

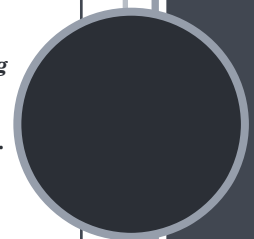
MODERNISERING AF KRITISK IT-INFRASTRUKTUR

Bachelor Projekt, EASV PB Software 2026

Dette bachelorprojekt præsenterer en strategisk og teknisk løsning for modernisering af legacy-infrastruktur i stærkt regulerede miljøer. Gennem implementeringen af en automatiseret GitOps-plattform demonstreres det, hvordan sikkerhed, compliance og agilitet kan forenes ved hjælp af Cloud-Native teknologier og Infrastructure as Code.

Klaus Hviid

05-01-2026



Resumé

Titel: Modernisering af kritisk it-infrastruktur

Forfatter: Klaus Hviid

Vejleder: Henrik Heinz Kühl

Dette bachelorprojekt adresserer udfordringen ved at modernisere it-infrastruktur i offentlige organisationer, hvor strenge krav til sikkerhed, compliance og netværksisolation ofte bremser overgangen til moderne teknologier. Med afsæt i erfaringer fra Skattestyrelsen undersøges det, hvordan man kan migrere fra imperativ, manuel drift til en deklarativ, automatiseret model uden at kompromittere integriteten af kritiske legacy-systemer.

Gennem metoden Design Science Research er der udviklet og implementeret en "Proof of Concept"-platform baseret på Cloud-Native principper. Løsningen anvender OpenTofu (Terraform) til infrastruktur-provisionering og Ansible til konfigurationsstyring for at etablere et High Availability Kubernetes-cluster (K3s). Som central styringsmekanisme implementeres GitOps ved hjælp af ArgoCD, hvilket sikrer fuld sporbarhed og versionsstyring af alle ændringer.

Projektet demonstrerer en hybrid arkitektur, der muliggør sikker interaktion med eksterne databaser. Ved at anvende en asynkron "Store-and-Forward" model for dataudveksling med skyen (Azure AKS), opnås en løsning, der respekterer datasuverænitet og sikrer ACID-transaktioner internt, samtidig med at skyens skalerbarhed udnyttes til datamodtagelse.

Konklusionen er, at en GitOps-baseret strategi effektivt kan erstatte traditionelle adgangsmetoder som "Jump Hosts" med auditerbare, rollebaserede godkendelses-workflows. Løsningen beviser, at det er muligt at opnå den agilitet, der kendetegner moderne softwareudvikling, inden for rammerne af et stærkt reguleret miljø, ved at flytte fokus fra perimetersikkerhed til processikkerhed.

MODERNISERING AF KRITISK IT-INFRASTRUKTUR

Indholdsfortegnelse

RESUMÉ.....	1
KAPITEL 1: INDLEDNING.....	4
1.1 BAGGRUND.....	4
1.2 PROBLEMFORMULERING.....	4
1.3 AFGRÆNSNING.....	5
1.4 KRAVSPECIFIKATION.....	5
1.4.1 Funktionelle Krav.....	6
1.4.2 Non-funktionelle Krav (Kvalitetskrav).....	6
1.5 METODE.....	7
1.6 PENSUMRELEVANS OG LÆRINGSMÅL.....	8
1.6.1 System Integration (F2025).....	8
1.6.2 Secure Software Development (E2024).....	8
1.6.3 Development of Large Systems (F2025).....	9
1.6.4 Databases for Developers (F2025).....	9
1.6.5 Software Quality (E2024).....	9
KAPITEL 2: TEORETISK FUNDAMENT.....	11
2.1 CLOUD-NATIVE PARADIGMET OG MIKROSERVICES.....	11
2.1.1 Fra Monolitisk til Mikroservice Arkitektur.....	11
2.1.2 Virtualisering vs. Containerisering.....	11
2.1.3 Container Orkestrering med Kubernetes.....	12
2.2 INFRASTRUCTURE AS CODE (IAC).....	12
2.2.1 Immutable vs. Mutable Infrastructure.....	12
2.2.2 Deklarativ vs. Imperativ Tilgang.....	12
2.2.3 Værktøjsvalg: Terraform og Ansible.....	12
2.3 GITOPS OG CONTINUOUS DELIVERY.....	13
2.3.1 GitOps Paradigmet.....	13
2.3.2 Push vs. Pull Deployment.....	13
2.4 SIKKERHED I CONTAINER-MILJØER.....	13
2.4.1 Netværkssegmentering og Isolation.....	13
2.4.2 Secrets Management.....	14
2.5 DATAARKITEKTUR: EKSTERNE AFHÆNGIGHEDER.....	14
2.5.1 Latens og Hybrid-arkitektur.....	14
2.5.2 Dataintegritet og ACID vs. CAP.....	14
2.6 OPSAMLING: TEORETISK RAMMEVÆRK OG PENSUM.....	15
KAPITEL 3: LØSNINGSARKITEKTUR.....	16
3.1 LØSNINGSARKITEKTUR.....	16
3.1.1 Infrastruktur-laget (Compute & Network).....	16
3.1.2 Platform-laget (Kubernetes).....	16
3.1.3 Applikations-laget (GitOps Engine).....	17
3.2 HYBRID CLOUD STRATEGI.....	17
3.2.1 Dataflow og Asynkron Integration.....	17
KAPITEL 4: IMPLEMENTERING.....	19
4.1 FASE 0: ETABLERING AF MANAGEMENT MILJØ (BOOTSTRAP).....	19
4.1.1 Admin Node Arkitektur.....	19
4.1.2 Klargøring af Værktøjskæden.....	19

4.1.3 Sikker Håndtering af SSH-identiteter ("Secret Zero").....	20
4.2 FASE 1: INFRASTRUKTUR SOM KODE (TERRAFORM).....	20
4.2.1 Modular Struktur og Providers.....	20
4.2.2 Udfordringer ved Cloud-Init og Storage.....	21
4.2.3 Dynamisk Inventory Generering.....	21
4.3 FASE 2: KONFIGURATION OG BOOTSTRAPPING (ANSIBLE).....	21
4.3.1 OS Hardening og Netværk (01-os-prep.yaml).....	21
4.3.2 K3s Cluster Etablering (02-k3s-install.yaml).....	21
4.3.3 ArgoCD Bootstrap (03-argocd-bootstrap.yaml).....	22
4.4 FASE 3: GITOPS MOTOREN (ARGOCD OG KUSTOMIZE).....	22
4.4.1 App of Apps Mønsteret.....	22
4.4.2 Miljødifferentiering med Kustomize.....	22
4.4.3 Release Gates.....	23
KAPITEL 5: SIKKERHEDSANALYSE OG COMPLIANCE.....	24
5.1 TRUSSELSBILLEDE OG RISIKOVURDERING.....	24
5.2 INTEGRITET: SPORBARHED OG IMMUTABLE INFRASTRUCTURE (MITIGERING AF T2).....	24
5.2.1 Git som Revisionslog.....	24
5.2.2 Godkendelses-workflows (Four-Eyes Principle).....	25
5.3 FORTROLIGHED: ISOLATION OG ADGANGSKONTROL (MITIGERING AF T3).....	25
5.3.1 Netværkssegmentering.....	25
5.3.2 Restriktiv Produktionsadgang.....	25
5.4 TILGÆNGELIGHED: SUPPLY CHAIN OG RESILIENCE (MITIGERING AF T1 & T4).....	26
5.4.1 Sikring af Supply Chain (Pull-modellen).....	26
5.4.2 Dataintegritet og Disaster Recovery.....	26
5.5 KONKLUSION PÅ SIKKERHEDSANALYSE.....	26
KAPITEL 6: DISKUSSION OG PERSPEKTIVERING.....	27
6.1 RESULTAT VS. PROBLEMFORMULERING.....	27
6.1.1 Vurdering af Strategiens Sikkerhed.....	27
6.1.2 Komplexitet ved High Availability (HA).....	27
6.2 REFLEKSION: ANSIBLE VS. GITOPS.....	28
6.2.1 Hvad har jeg lært?.....	28
6.2.2 Det Hybride Dilemma.....	28
6.3 SKALERING TIL AZURE (HYBRID CLOUD).....	28
6.3.1 Databaser og CAP-teoremet.....	29
6.3.2 Data-suverænitet.....	29
6.4 HVAD MANGLER? (FUTURE WORK).....	29
6.5 KONKLUSION PÅ DISKUSSION.....	30
KAPITEL 7: KONKLUSION.....	31
KILDE- OG LITTERATURLISTE:.....	32
BØGER.....	32
STANDARDE OG RAPPORTER.....	32
OFFICIEL DOKUMENTATION (TEKNISK IMPLEMENTERING).....	33
ARTIKLER OG ØVRIGE KILDER.....	33
BILAG:.....	35
REPOSITORY OG REPRODUCERBARHED:.....	35

Kapitel 1: Indledning

1.1 Baggrund

Digitaliseringen af den offentlige sektor i Danmark har gennem det seneste årti undergået en markant transformation. Hvor fokus tidligere lå på digitalisering af enkelte arbejdsgange, er dagsordenen nu skiftet mod etablering af sammenhængende, datadrevne platforme, der kan understøtte agil udvikling og hurtig tilpasning til lovgivningsmæssige ændringer.

Denne transformation møder dog betydelig modstand i form af teknisk gæld. Store offentlige instanser, herunder Skattestyrelsen, driver kritiske samfundssystemer på legacy-infrastruktur, hvor stabilitet og sikkerhed har højeste prioritet. Disse miljøer er typisk karakteriseret ved stærke netværkssegmenteringer og tunge manuelle driftsprocesser.

En central udfordring i denne infrastruktur er afhængigheden af monolitiske database-arkitekturer. I denne kontekst refererer begrebet ikke blot til databasesoftware, men til en arkitektonisk tilstand, hvor én enkelt, omfattende databaseinstans fungerer som det centrale datalager for adskillige forretningsdomæner og it-systemer. Disse databaser er ofte placeret på tunge Storage Area Networks (SAN) og skalerer vertikalt, hvilket skaber en stærk kobling mellem systemerne og udgør en betydelig barriere for modernisering.

Udfordringen opstår i spændingsfeltet mellem kravet om modernisering – herunder ønsket om at udnytte teknologier, der understøtter en cloud-native arkitektur, herunder containere og mikroservices [Kilde 7] – og de ufravigelige krav om compliance og sikkerhed i et stærkt reguleret miljø. Hvordan introducerer man modernitet i et system, der ikke må fejle, og hvor internetadgang er stærkt begrænset?

Dette bachelorprojekt tager afsæt i konkrete erfaringer fra drift af sådanne miljøer, hvor imperativ automatisering via Ansible¹ har været den primære driftsform. Projektet undersøger, hvordan et skift til en deklarativ GitOps-model² kan løse de sikkerhedsmæssige og driftsmæssige udfordringer, der er forbundet med at drive en moderne containerplatform under restriktive forhold.

1.2 Problemformulering

På baggrund af ovenstående tager projektet udgangspunkt i følgende problemformulering:

Modernisering af kritisk it-infrastruktur

1 - Ansible: Et open-source IT-automatiseringsværktøj til konfigurationsstyring (Configuration Management), applikationsdeployment og opgavestyring. Ansible er kendetegnet ved at være agent-løst og opererer typisk imperativt (udfører sekventielle kommandoer) via SSH.

2 - GitOps: En operationel model for cloud-native applikationer, hvor Git-versionsstyring fungerer som "Single Source of Truth". I modsætning til traditionel CI/CD anvender GitOps en "Pull-baseret" tilgang, hvor en agent inde i clusteret (f.eks. ArgoCD) automatisk synkroniserer infrastrukturen, så den matcher koden i Git.

Denne opgave omhandler, hvordan en omfattende og sikker strategi for platformmodernisering og migrering af legacy-applikationer til cloud-native containerorkestreringsplatforme kan formuleres og eksekveres i et stærkt reguleret, dataintensivt og sikkerhedsbevidst offentligt sektormiljø.

Med afsæt i erfaringer fra Skattestyrelsen, hvor deployment via Ansible var den eneste mulighed grundet fuldstændigt adskilte netværk og særlige sikkerhedshensyn, og hvor en ekstern, SAN-baseret PostgreSQL-database udgjorde en central, men eksternt styret, afhængighed, vil jeg dybdegående analysere:

- Kritisk planlægning og eksekvering af migreringer for at sikre minimal nedetid og dataintegritet, især i lyset af eksterne databaseafhængigheder.
- Integration af strenge sikkerheds- og revisionskrav gennem hele migrationsprocessen og i den resulterende cloud-native arkitektur.
- Valg og optimering af automatiseringsværktøjer under de givne restriktive forhold, hvor processen var "nyt for alle parter".

Projektet sigter mod at identificere best practices og "lessons learned" fra et retrospektivt perspektiv for at guide fremtidige transformationsprojekter i komplekse og delvis kontrollerede it-landskaber.

1.3 Afgrænsning

For at sikre dybde og fokus i besvarelsen er følgende afgrænsninger foretaget:

- **Applikationskode:** Projektet fokuserer på platformen og deployment-processen. Der foretages ikke omskrivning (refactoring) af selve legacy-applikationens kildekode til mikroservices, men derimod en "Lift and Shift" til containere for at demonstrere platformens kapabiliteter.
- **Netværkshardware:** Netværkssegmentering simuleres logisk via VLANs og firewalls i virtualiseringslaget (Proxmox/SDN). Konfiguration af fysiske routere, switches og hardware-firewalls ligger uden for opgavens scope.
- **Database-optimering:** Fokus er på forbindelsen og sikkerheden omkring den eksterne database, ikke på performance-tuning af selve PostgreSQL-instansen eller SQL-forespørgsler.
- **Specifik Jura:** Selvom compliance (GDPR/Schrems II) berøres i forhold til data-suverænitet og hybrid cloud, udarbejdes der ikke en fuld juridisk risikoanalyse. Fokus er på de tekniske kontroller, der understøtter compliance.

1.4 Kravspecifikation

For at sikre, at platformen kan understøtte både modernisering af legacy-applikationer og overholdelse af strenge sikkerhedskrav, er følgende krav identificeret som kritiske for løsningen ("Must-haves").

1.4.1 Funktionelle Krav

Systemet skal kunne:

Understøtte fuld livscyklusstyring: Platformen skal facilitere en komplet release-pipeline fra udvikling til produktion gennem dedikerede miljøer, der understøtter en struktureret teststrategi jævnfør principperne for Continuous Integration [Kilde 5]:

- **Udvikling:** DEV.
- **Verifikation:**
 - **TST** (Unit testing).
 - **SIT** (System Integration Testing).
 - **FKT** (Funktionstest/System Testing).
- **Produktion & Partnersamarbejde:**
 - **TFE** (Test for Erhverv - Acceptance Test/UAT udført med eksterne partnere).
 - **PRD** (Live produktion).

Håndtere hybrid data-tilgang: Applikationer skal kunne konfigureres til at anvende interne, flygtige databaser i testmiljøer og eksterne, persistente databaser (SAN) i produktionsmiljøer uden ændringer i kildekoden. Dette understøtter kravet om at bevare transaktionel integritet (ACID) for produktionsdata [Kilde 1].

Automatisere infrastruktur: Provisionering af nye noder og miljøer skal ske automatisk uden manuel interaktion via SSH (Infrastructure as Code), for at minimere menneskelige fejl og sikre ensartethed [Kilde 3].

1.4.2 Non-funktionelle Krav (Kvalitetskrav)

Sikkerhed og Compliance:

- **Netværkssegmentering:** Trafik mellem applikationer og databaser skal være isoleret fra management-trafik for at opfylde princippet om "Defense in Depth" [Kilde 6].
- **Audit Trail:** Alle ændringer i infrastrukturen og applikationerne skal være sporbare ned til specifik bruger og tidspunkt (GitOps).
- **Ingen direkte adgang:** Udviklere må ikke have direkte skriveadgang til produktionsclusteret.

High Availability (HA): Platformen må ikke have "Single Point of Failure". Kontrolplanet (Kubernetes Master) skal være redundant, og workloads skal kunne fordeles på tværs af fysiske noder (X-akse skalering i The Scalability Cube) [Kilde 2].

Portabilitet (Hybrid Cloud): Løsningen skal være agnostisk over for den underliggende hardware, således at workloads kan flyttes fra on-premise (Proxmox) til public cloud (Azure) med minimal rekonfiguration [Kilde 7].

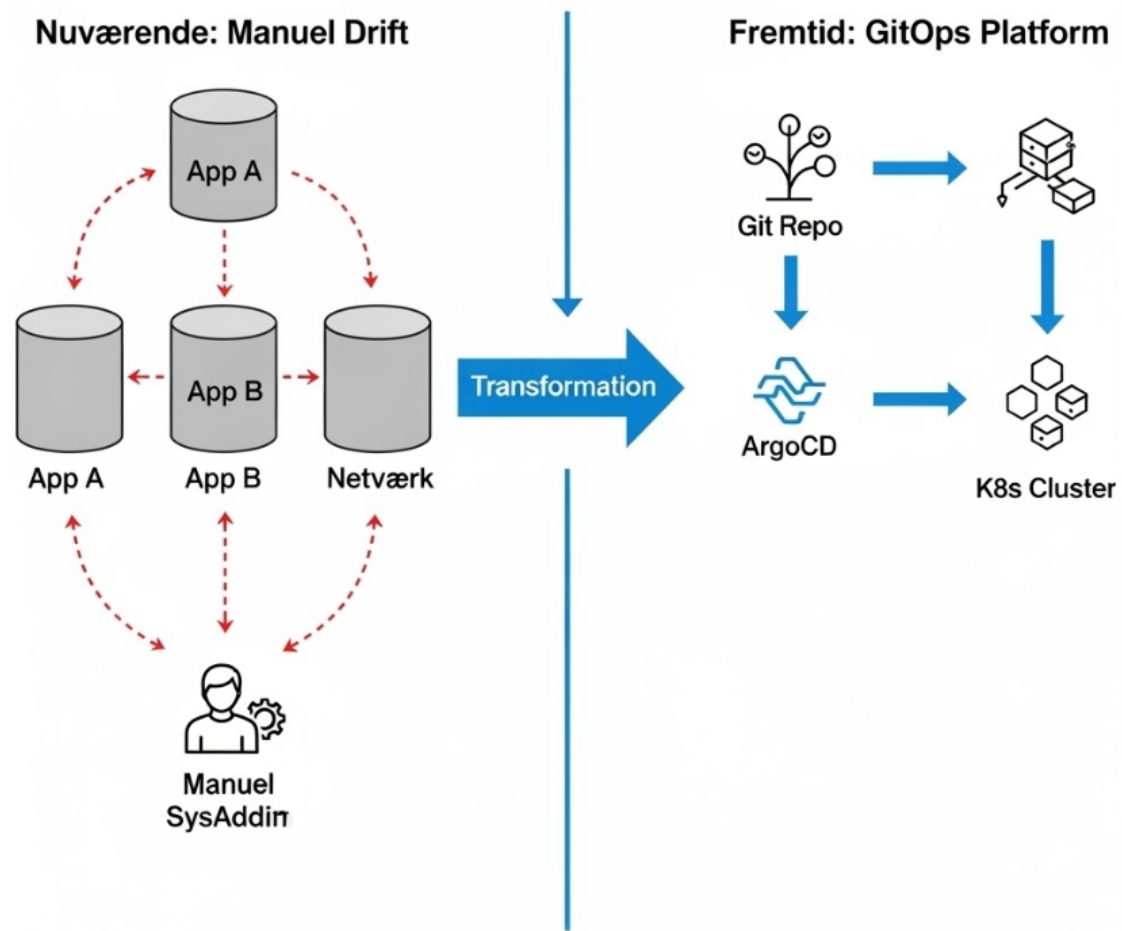
1.5 Metode

Projektet anvender Design Science Research (DSR)³ som metodisk rammeværk. DSR er velegnet til it-projekter, hvor målet er at løse et praktisk problem gennem udviklingen af en "artefakt" – i dette tilfælde den automatiserede GitOps-platform.

Tilgangen er iterativ og adskiller sig fra en lineær vandfaldsmodel. Projektet gennemløber flere cyklusser, hvor erfaringer fra Demonstration og Evaluering fører tilbage til justeringer i Design og Udvikling. Artefaktet modnes således gennem følgende aktiviteter:

1. **Problemanalyse (Explicate Problem):** Kortlægning af kravene til sikkerhed og drift i regulerede miljøer baseret på erfaringer og litteraturstudier.
2. **Løsningsdesign (Define Requirements):** Udformning af en arkitektur, der kombinerer Infrastructure as Code (Terraform) og GitOps (ArgoCD) for at imødekomme kravene om sporbarhed og netværksisolation.
3. **Design og Udvikling (Design & Develop Artifact):** Implementering af "Proof of Concept" (PoC) platformen på Proxmox hardware. Denne fase inkluderer iterativ udvikling af Ansible playbooks, Terraform moduler og Kubernetes manifeste baseret på løbende tests.
4. **Demonstration (Demonstrate Artifact):** Validering af løsningen gennem simulerede scenarier, herunder deployment fra udvikling til produktion og disaster recovery. Fejl fundet her fører til nye iterationer af udviklingsfasen.
5. **Evaluering (Evaluate Artifact):** Kritisk analyse af løsningens effektivitet i forhold til problemformuleringen, herunder sammenligning mellem den imperative (Ansible) og deklarative (GitOps) tilgang.

³ -Design Science Research (DSR): En forskningsmetodologi, der fokuserer på skabelsen og evalueringen af innovative artefakter (f.eks. software, modeller eller metoder) for at løse praktiske problemer. DSR adskiller sig fra traditionel naturvidenskabelig forskning ved at være løsningsorienteret og iterativ, hvor ny viden genereres gennem selve design- og konstruktionsprocessen frem for blot observation.



1.6 Pensumrelevans og Læringsmål

Projektet demonstrerer anvendelse af teori og metoder fra uddannelsens kernemoduler, med særligt fokus på følgende læringsmål:

1.6.1 System Integration (F2025)

Projektet adresserer direkte læringsmålene vedrørende "Managing a Kubernetes cluster & deploying microservices".

- **Microservice Architecture:** Implementeringen af K3s og ArgoCD demonstrerer en moderne platform til orkestrering af services, i modsætning til monolitisk drift.
- **Secrets Management:** Håndtering af "Certificates, private keys and API keys" demonstreres gennem brugen af Kubernetes Secrets og diskussionen om sikker opbevaring i GitOps (Sealed Secrets).
- **Deployment of Microservices:** Automatisering af deployment-processen via "Infrastructure as Code" og containerisering.

Implementering i projektet:

Der er etableret et High Availability (HA) Kubernetes-cluster bestående af 2 Masters og 2 Workers. Deployment-processen er fuldt automatiseret via Ansible-playbooks (02-k3s-install.yaml), der håndterer den komplekse token-udveksling og join-logik mellem noderne, samt sikrer korrekt TLS-konfiguration (SANs) for API-serveren.

1.6.2 Secure Software Development (E2024)

Løsningen er designet med "Security by Design" som omdrejningspunkt, jf. OWASP Proactive Controls.

- **Supply Chain Security:** Ved at implementere en "Pull-baseret" GitOps-model med et internt Git-repository minimeres risikoen for Supply Chain angreb, da deployment-motoren (ArgoCD) aldrig eksponeres mod internettet.
- **Access Control (RBAC):** Projektet implementerer streng adgangskontrol, hvor udviklere ingen direkte adgang har til produktionsmiljøet, hvilket erstatter traditionel perimetersikkerhed.

Implementering i projektet:

Sikkerhed implementeres ved at fjerne behovet for indgående adgang til clusteret. ArgoCD, der kører inde i clusteret, trækker ændringer fra Git. Adgang til administrationsnoden (b-admin) er sikret via SSH-nøgler injiceret gennem Terraform's Cloud-Init konfiguration, og netværket er segmenteret i et offentligt (192.168.8.x) og et privat (10.0.0.x) plan for at isolere kontroltrafik.

1.6.3 Development of Large Systems (F2025)

- **The Scalability Cube:** Projektet demonstrerer skalering langs X-aksen (Horizontal Duplication) ved at anvende Kustomize overlays til at definere antallet af replikaer i produktionsmiljøet for at opnå High Availability.
- **DevOps:** Hele løsningen er en manifestation af DevOps-kulturen, hvor kløften mellem udvikling (Code) og drift (Infrastructure) lukkes gennem automatisering og fælles værktøjer.

Implementering i projektet:

Skalerbarhed demonstreres konkret i kubernetes/overlays/prd/kustomization.yaml, hvor en Kustomize patch ændrer replicas fra 1 til 3. Hele infrastrukturen styres som kode via OpenTofu (Terraform), der muliggør skalering af den underliggende hardware (flere worker-noder) ved blot at ændre en variabel i variables.tf og køre tofu apply.

1.6.4 Databases for Developers (F2025)

- **Transactions & ACID:** Valget af en hybrid strategi, hvor produktionsdata fastholdes på eksterne SAN-systemer (PostgreSQL), er truffet for at garantere ACID-egenskaber (Atomicity, Consistency, Isolation, Durability), som kan være udfordret i rent distribuerede storage-løsninger.
- **Distributed Databases & CAP:** I diskussionen om Hybrid Cloud (Azure vs. On-prem) analyseres afvejningen mellem Consistency og Availability i henhold til CAP-teoremet ved netværkslatens.

Implementering i projektet:

Datastrategien realiseres gennem en hybrid konfiguration. I udviklingsmiljøet (DEV) deployeres en Postgres StatefulSet container automatisk sammen med applikationen. I produktionsmiljøet (PRD) konfigureres applikationen via Secrets til at forbinde til en ekstern, persistent database-instans. Trafikken til denne database dirigeres over det dedikerede 10-netværk for at minimere latens og sikre isolation.

1.6.5 Software Quality (E2024)

- **Continuous Integration:** Etableringen af en struktureret pipeline danner infrastrukturen for en "Agile Testing Strategy", der muliggør automatiserede tests på alle niveauer før produktion.

Implementering i projektet:

Kvalitetssikring er indbygget i miljøstrukturen defineret i all-envs.yaml. Pipeline-flowet er designet til at promovere kode gennem distinkte faser: DEV (Udvikling) -> TST (Unit Test) -> SIT (Integration) -> FKT (Funktionstest) → TFE (Partner/ReadOnly-Prod/UserAcceptanceTest) -> PRD (Produktion). ArgoCD sikrer, at hvert miljø præcist afspejler den testede kodeversion.

Kapitel 2: Teoretisk fundament

2.1 Cloud-Native Paradigmet og Mikroservices

For at forstå nødvendigheden af platformmodernisering i offentlige instanser som Skattestyrelsen, er det essentielt først at definere skiftet fra traditionel virtualisering til cloud-native teknologier. Dette afsnit redegør for de fundamentale koncepter bag containere, orkestrering og mikroservice-arkitektur, som behandlet i faget System Integration (F2025).

2.1.1 Fra Monolitisk til Mikroservice Arkitektur

Legacy-systemer er historisk set ofte bygget som monolitter. En monolitisk applikation er karakteriseret ved, at al funktionalitet – brugergrænseflade, forretningslogik og dataadgang – er pakket i én samlet enhed, der kører i én proces på serveren [Kilde 1].

Selvom monolitter er simple at udvikle i starten, introducerer de betydelige udfordringer ved skalering og vedligeholdelse:

- **Kobling:** En ændring i én del af koden kan kræve genkompilering og genstart af hele systemet.
- **Skalering:** Man er tvunget til at skalere hele applikationen (X-akse skalering i The Scalability Cube), selvom det kun er én funktion, der oplever høj belastning.
- **Teknologilåsning:** Det er vanskeligt at indføre nye teknologier i en eksisterende monolit.

Som svar på disse udfordringer er mikroservice-arkituren opstået. Her opdeles applikationen i små, uafhængige tjenester, der kommunikerer via veldefinerede API'er. Dette muliggør, at enkelte komponenter kan udvikles, deployes og skaleres uafhængigt af hinanden.

2.1.2 Virtualisering vs. Containerisering

For at understøtte mikroservices har infrastrukturen ændret sig markant.

- **Virtuelle Maskiner (Hardware Virtualisering):** En VM emulerer en komplet fysisk computer med eget fuldt OS. Dette giver stærk isolation, men medfører overhead [Kilde 10].
- **Containere (OS Virtualisering):** Containere pakker koden og dens afhængigheder, men deler værtsmaskinens Linux-kerne. De er letvægts og portable [Kilde 7].

I en moderniseringsstrategi er dette skift kritisk: Hvor VM'er ("Pet"-mentalitet) ofte opdateres manuelt, er containere ("Cattle"-mentalitet) immutable; man opdaterer ikke en kørende container, man erstatter den [Kilde 3].

2.1.3 Container Orkestrering med Kubernetes

Med skiftet til mikroservices opstår behovet for orkestrering. Kubernetes (K8s) tilbyder en deklarativ tilgang til drift, hvor man definerer en "ønsket tilstand" (Desired State). Kubernetes' kontrolplan overvåger systemet og sikrer via "Self-healing", at den faktiske tilstand matcher den ønskede [Kilde 2].

I dette projekt anvendes K3s, en CNCF-certificeret letvægtsdistribution, der fjerner legacy-komponenter, hvilket gør den ideel til on-premise miljøer [Kilde 11].

2.2 Infrastructure as Code (IaC)

For at realisere agiliteten i cloud-native arkitekturen, er det nødvendigt at ændre måden, hvorpå infrastrukturen styres. Dette understøttes af principperne fra faget Development of Large Systems (F2025) omkring DevOps.

2.2.1 Immutable vs. Mutable Infrastructure

I traditionel drift ("Mutable") opdateres servere løbende manuelt, hvilket fører til "Configuration Drift", hvor servere divergerer over tid. IaC fremmer "Immutable Infrastructure", hvor en server aldrig ændres efter deployment. Ved ændringer erstattes den med en ny [Kilde 3]. I dette projekt sikrer Terraform immutabilitet på VM-niveauet, mens Ansible sikrer konsistens på OS-niveauet.

2.2.2 Deklarativ vs. Imperativ Tilgang

- **Imperativ (Procedural):** Fokuserer på hvordan en ændring skal udføres (scripts/Ansible).
- **Deklarativ (Funktionel):** Fokuserer på hvad slutresultatet skal være (Terraform/Kubernetes) [Kilde 9].

2.2.3 Værktøjsvalg: Terraform og Ansible

- **OpenTofu (Terraform):** Anvendes til provisionering af hardwaren (VM'er). Det styrer tilstand (State) og afhængigheder mellem ressourcer [Kilde 9].

- **Ansible:** Anvendes til konfiguration af OS og installation af Kubernetes. Det er valgt til "bootstrapping"-fasen, da det mestrer den sekventielle, imperative logik, der kræves for at gøre en rå server klar til drift [Kilde 8].

2.3 GitOps og Continuous Delivery

Dette afsnit omhandler metoden for applikationsleverance, med direkte reference til fagene Software Quality (E2024) og Secure Software Development (E2024).

2.3.1 GitOps Paradigmet

GitOps anvender Git som "Single Source of Truth". Kerneprincippet er: Hvis det ikke står i Git, eksisterer det ikke. Dette sikrer versionering og audit-trails for hele platformen [Kilde 4, 17].

2.3.2 Push vs. Pull Deployment

Et kritisk sikkerhedsvalg er skiftet fra Push til Pull.

- **Push-modellen (CI/CD):** En ekstern CI-server har admin-adgang til clusteret og "skubber" ændringer. Dette udgør en sikkerhedsrisiko i regulerede miljøer, da det kræver indgående adgang [Kilde 5].
- **Pull-modellen (GitOps/ArgoCD):** En agent (ArgoCD) inde i clusteret "trækker" ændringer fra Git. Clusteret behøver ingen åbne indgående porte til management, hvilket minimerer angrebsfladen markant [Kilde 4, 12].

Intern Source Control (Supply Chain Security) I scenarier med interne Git-repositories (som i Skattestyrelsen) opnås maksimal sikkerhed ved brug af Pull-modellen. Da ArgoCD befinder sig inde i clusteret og trækker konfigurationen fra det interne netværk, eksponeres deployment-processen aldrig mod internettet, hvilket imødegår "Supply Chain Attacks" [Kilde 12].

2.4 Sikkerhed i Container-miljøer

Med afsæt i faget Secure Software Development (E2024) etableres fundamentet for sikkerhed.

2.4.1 Netværkssegmentering og Isolation

Løsningen implementerer "Defense in Depth" gennem streng netværkssegmentering [Kilde 6]:

- **Management Plane (Public):** Dedikeret til administration.
- **Control Plane / Data Plane (Private):** Isoleret netværk til intern replikering og database-trafik.

Egress Control: For at imødekomme krav om sikker interaktion med eksterne systemer, anvendes principper for Egress Filtering, hvor kun specifikke pods har adgang til de eksterne SAN-databaser [Kilde 6].

Restriktiv Produktionsadgang (Erstatning af Jump Hosts) I legacy-miljøet styres adgang til produktion via fysiske SSH "Jump Hosts". I GitOps-løsningen erstattes dette af ArgoCD's Manuelle Gates. Adgangen til at synkronisere ændringer til PRD begrænses via RBAC, hvilket fungerer som en digital, auditerbar gatekeeper, der erstatter behovet for fysisk serveradgang [Kilde 12].

2.4.2 Secrets Management

Håndtering af hemmeligheder i Git løses teoretisk via Kubernetes Secrets, men i en produktionskontekst kræves løsninger som Sealed Secrets eller External Secrets (HashiCorp Vault) for at sikre, at hemmeligheder er krypteret i hvile og i versionsstyring [Kilde 13].

2.5 Dataarkitektur: Eksterne Afhængigheder

Interaktionen med eksterne databaser behandles med afsæt i Databases for Developers (F2025).

2.5.1 Latens og Hybrid-arkitektur

Når applikationslaget (Azure/Kubernetes) separeres fra datalaget (On-premise SAN), introduceres netværkslatens. Løsningen optimerer dette ved dedikerede 10-netværksforbindelser og Kubernetes Services (ExternalName), der abstraherer den fysiske placering for applikationen [Kilde 1].

2.5.2 Dataintegritet og ACID vs. CAP

I et distribueret hybrid-miljø udfordrer CAP-teoremet (Consistency, Availability, Partition Tolerance) systemdesignet [Kilde 15].

- **PRD (Internt):** For data i den centrale database (skattemappen) prioriteres Konsistens (ACID) ubetinget.
- **Hybrid Integration (Eksternt):** For integrationen med eksterne cloud-services accepteres en model baseret på Eventual Consistency. Data modtages i skyen (Høj Tilgængelighed/Availability) og transporteres asynkront til det interne miljø. Dette sikrer, at systemet kan modtage data selv ved midlertidige netværksudfald mod det interne netværk, men data er først konsistente i kernen efter behandling.

2.6 Opsamling: Teoretisk Rammeværk og Pensum

Nedenstående tabel sammenfatter, hvordan teori fra undervisningen anvendes til at løse de specifikke problemer defineret i problemformuleringen:

Problemstilling	Teoretisk Koncept	Teknisk Løsning	Relevant Fagmodul	Branche
Platformmodernisering	Mikroservices, Containerisering.	K3s (Kubernetes): Abstraherer applikationen fra hardwaren.	System Integration (F2025)	Cloud Architecture / Platform Engineering
Sikkerhed & Compliance	Network Policies, RBAC, Secure by Design.	Privat Netværk & Namespace Isolation.	Secure Software Development (E2024)	IT Security / DevSecOps
Sikker Deployment	GitOps (Pull-model), CI/CD.	ArgoCD + Intern Git: Erstatte Jump Hosts.	Software Quality (E2024) / Dev. of Large Systems (F2025)	DevOps / Release Management
Eksterne Afhængigheder	ACID, CAP-teorem, Latens.	Hybrid Overlay: PRD mod ekstern DB, DEV mod intern.	Databases for Developers (F2025)	Database Administration / Data Architecture

Kapitel 3: Løsningsarkitektur

Med udgangspunkt i de identificerede krav i Kapitel 1 præsenterer dette kapitel det valgte løsningsdesign og den overordnede systemarkitektur. Arkitekturen er designet til at understøtte en sikker, skalerbar og automatiseret platformmodernisering.

3.1 Løsningsarkitektur

Baseret på kravene er der designet en lagdelt arkitektur, der kombinerer Infrastructure as Code (IaC) med GitOps.



3.1.1 Infrastruktur-laget (Compute & Network)

I bunden af stakken anvendes Proxmox VE som hypervisor til at simulere et on-premise datacenter. For at imødekomme sikkerhedskravet om segmentering er netværket opdelt i to fysisk adskilte VLANs:

- **Public/Mgmt Network (192.168.8.0/24):** Håndterer indgående trafik (Ingress), SSH-adgang til administration og API-kald fra Terraform.
- **Cluster Private Network (10.0.0.0/24):** Et lukket netværk dedikeret til intern replikering mellem Kubernetes-noder (etcd), storage-trafik og databaseforbindelser. Dette sikrer, at følsom replikeringstrafik ikke kan aflyttes eller tilgås fra det offentlige netværk [Kilde 6].

Provisioneringen af dette lag styres deklarativt af OpenTofu (Terraform), som sikrer, at antallet af noder og deres ressourcer (CPU/RAM/Disk) altid matcher definitionen i koden [Kilde 9].

3.1.2 Platform-laget (Kubernetes)

Ovenpå infrastrukturen etableres container-orkestreringslaget ved hjælp af K3s. Valget af K3s frem for "vanilla" Kubernetes skyldes dets letvægtsnatur og reducerede angrebsflade, hvilket er fordelagtigt i edge-lignende on-premise miljøer [Kilde 11].

Clusteret er konfigureret i en High Availability (HA) topologi for at sikre driftstabilitet:

- **2 Master Noder:** Kører API-server og etcd-database.
- **2 Worker Noder:** Dedikeret til afvikling af applikations-workloads.
- **Node labels og Taints:** Anvendes til at styre placeringen af workloads (Scheduling), således at kritiske system-pods holdes adskilt fra applikations-pods, og at applikationer fordeles på tværs af fysiske værter for at sikre opetid ved hardwarefejl [Kilde 2].

Konfigurationen og hærddningen af dette lag (OS prep, firewall, binær installation) udføres imperativt og idempotent af Ansible [Kilde 8].

3.1.3 Applikations-laget (GitOps Engine)

Som "Single Pane of Glass" for alle applikationer og miljøer anvendes ArgoCD. ArgoCD fungerer som en Kubernetes Controller, der overvåger Git-repository'et og realiserer Pull-modellen [Kilde 4].

Ved at anvende Kustomize struktureres applikationerne i en Base (fælles kode) og Overlays (miljøspecifik konfiguration) [Kilde 13]. Dette design muliggør den hybride data-strategi:

- I Base defineres applikationen til at forvente en database-forbindelse via en miljøvariabel (12-Factor App princip) [Kilde 16].
- I DEV overlayet injiceres en forbindelse til en intern container-database.
- I PRD overlayet injiceres en forbindelse til den eksterne SAN-database, hvilket sikrer data-persistens uden for container-plattformen.

3.2 Hybrid Cloud Strategi

Et centralt element i moderniseringsstrategien er muligheden for at udnytte Public Cloud (Azure) til skalerbarhed, mens følsomme data eller legacy-systemer fastholdes on-premise.

Arkitekturen understøtter dette ved at afkoble Control Plane (ArgoCD) fra Data Plane (Kubernetes Clustere).

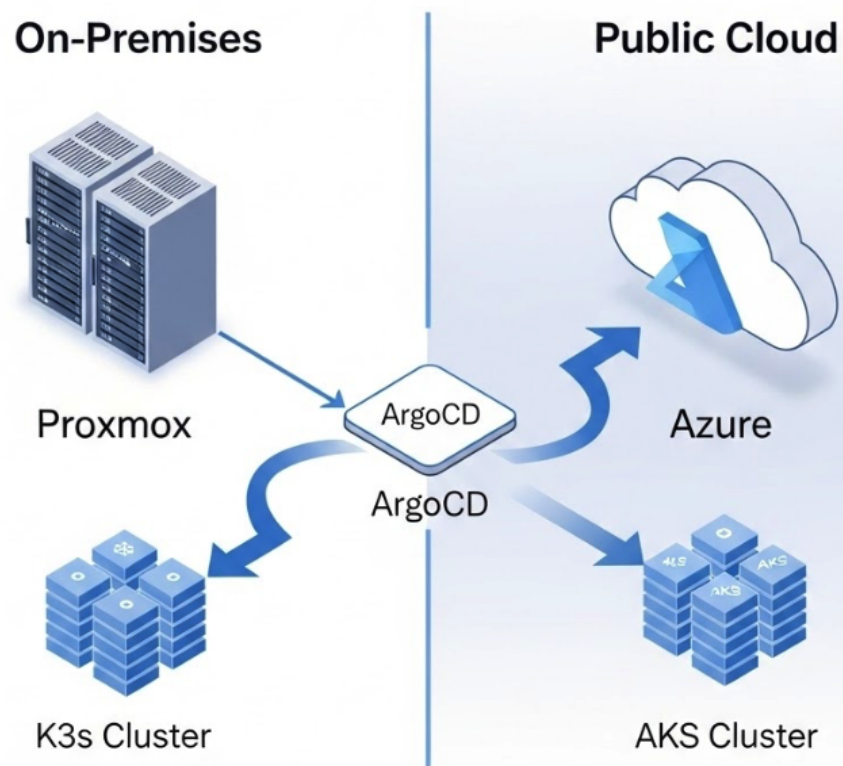
- **Central Styring:** ArgoCD kører on-premise (i det sikre miljø) og agerer som central kontrolenhed.
- **Distribueret Eksekvering:** Produktionsmiljøet (PRD) kan defineres som et target i Azure Kubernetes Service (AKS) [Kilde 14]. Da ArgoCD kun kræver adgang til Kubernetes API'et (og ikke omvendt), kan deployment til skyen styres sikkert fra det lukkede netværk.

3.2.1 Dataflow og Asynkron Integration

For at imødekomme kravene om datasuverænitet og sikkerhed anvendes en asynkron integrationsmodel i hybrid-scenariet. I stedet for direkte databaseadgang fungerer cloud-komponenterne som "Ingress Points" for eksterne data (f.eks. bank-XML).

1. **Modtagelse:** Applikationen i Azure modtager data.
2. **Transport:** Data videresendes via krypteret forbindelse til et internt mellemlager (Staging Area).
3. **Behandling:** Interne processer vasker og validerer data før lagring i den centrale database (Skattemappen).

Dette design sikrer, at den centrale database aldrig eksponeres mod skyen, og at systemet opnår høj tilgængelighed (Availability) for datamodtagelse, selvom forbindelsen til det interne system midlertidigt skulle være nede (Eventual Consistency).



Kapitel 4: Implementering

Dette kapitel dokumenterer den tekniske realisering af løsningsarkitekturen. Implementeringen er opdelt i fire faser, der tilsammen udgør en automatiseret "Zero-to-Hero" pipeline: Bootstrap af management, Infrastruktur (Provisioning), Konfiguration (Bootstrapping) og Applikationsstyring (GitOps).

Afsnittet beskriver de konkrete tekniske valg, konfigurationsdetaljer og løsninger på de udfordringer, der opstod under udviklingen.

4.1 Fase 0: Etablering af Management Miljø (Bootstrap)

Før automatiseringen kan begynde, skal der etableres et sikkert og kontrolleret udgangspunkt. I stedet for at køre automationen direkte fra en udvikler-laptop, hvilket kan introducere afhængigheder af lokale miljøvariabler og OS-specifikke værktøjer, blev der etableret en dedikeret Management Node (`b-admin`).

4.1.1 Admin Node Arkitektur

`b-admin` fungerer som det centrale Control Plane for hele infrastrukturen.

- **Rolle:** Den agerer som "Jump Host" og eksekveringsmiljø for Terraform og Ansible.
- **Placering:** Placeret på management-netværket (192.168.8.100) med adgang til Proxmox API'et og GitHub, men uden direkte adgang til clusterets interne datatrafik.
- **Sikkerhed:** Adgang til noden er sikret via SSH-nøgler, og den indeholder de nødvendige API Tokens til Proxmox og Azure.

4.1.2 Klargøring af Værktøjskæden

For at sikre reproducerbarhed og fuld sporbarhed blev opsætningen af selve Admin Noden scriptet i `install_tools.sh`. Dette script opbevares i Git-repository'et, hvilket sikrer en audit trail for ændringer i værktøjsversioner. Dette sikrer samtidig, at hvis noden fejler, kan en ny spinnes op og klargøres på få minutter med præcis samme versioner af værktøjerne [Kilde 3]:

1. **OpenTofu:** Til infrastruktur-provisionering [Kilde 9].
2. **Ansible:** Til konfigurationsstyring [Kilde 8].
3. **Kubectl & Azure CLI:** Til interaktion med Kubernetes og skyen.
4. **Git:** Til versionsstyring og hentning af kodebasen.

Denne tilgang løser "Hønen og ægget"-problemet ved at definere et minimalt manuelt startpunkt, hvorfra resten af infrastrukturen kan rulles ud automatisk.

4.1.3 Sikker Håndtering af SSH-identiteter ("Secret Zero")

En central sikkerhedsbeslutning i designet er håndteringen af de kryptografiske nøgler. Mens Cloud-Init automatisk installerer den offentlige nøgle på `b-admin` for at tillade indgående administrator-adgang, distribueres den private nøgle (`id_bachelor_project`) bevidst ikke automatisk.

Da `b-admin` fungerer som kontrolenhed, har den behov for at besidde nøgleparret fysisk i filsystemet:

1. **Privat Nøgle:** Nødvendig for at `b-admin` kan autentificere sig over for GitHub (ved `git clone`) og etablere SSH-forbindelser til de øvrige noder via Ansible.
2. **Offentlig Nøgle:** Nødvendig for at Terraform kan indlæse nøglen dynamisk og injicere den i de nye Kubernetes-noder, der provisioneres.

For at undgå at gemme den private nøgle i ukrypterede scripts, templates eller Git (hvilket ville bryde sikkerhedsprincipperne), anvendes en manuel "Out-of-Band" overførsel via SCP (Secure Copy) som det allerførste trin efter oprettelsen af Admin Noden. Dette sikrer, at den private nøgle kun eksisterer på administratorens lokale maskine og på den sikrede Admin Node.

4.2 Fase 1: Infrastruktur som Kode (Terraform)

Fundamentet for platformen er skabt ved hjælp af OpenTofu (Terraform). Målet med denne fase var at transformere den manuelle oprettelse af virtuelle maskiner på Proxmox til en reproducerbar kodebase.

4.2.1 Modulær Struktur og Providers

Koden er struktureret for at adskille logik fra konfiguration.

- **providers.tf:** Definerer forbindelsen til Proxmox API'et (`telmate/proxmox`). Her anvendes API Tokens med begrænsede rettigheder for at undgå brug af root-password, hvilket imødekommer sikkerhedskravene [Kilde 10].
- **main.tf:** Indeholder ressource-definitionerne. Ved at anvende `for_each` løkker itererer Terraform over en liste af noder (defineret i `variables.tf`), hvilket gør det trivielt at skalere clusteret fra 2 til 10 noder ved blot at ændre én variabel.

4.2.2 Udfordringer ved Cloud-Init og Storage

En central udfordring i implementeringen var håndteringen af Cloud-Init på tværs af forskellige storage-typer (`local-zfs` vs `vm-storage`). Standard Proxmox-skabeloner kræver et Cloud-Init drev for at injicere SSH-nøgler og IP-adresser ved første boot. Løsningen blev en eksplicit definition af `cicustom` og disk-slots i Terraform, der sikrer, at OS-disken klones korrekt fra skabelonen, mens netværkskonfigurationen injiceres dynamisk.

For at løse problemet med "Boot Hangs" (hvor VM'en venter på entropi eller disk-resize) blev der implementeret specifikke cpu typer (`x86-64-v2-AES`) og `virtio-scsi` controllere, der sikrer optimal I/O-performance til Kubernetes-workloads.

4.2.3 Dynamisk Inventory Generering

For at binde Terraform sammen med det efterfølgende Ansible-lag, blev der udviklet en automatiseret inventory-generator (`inventory.tf`). Ved hjælp af `local_file` ressourcen og en template-fil (`inventory.tftpl`), skriver Terraform automatisk en `inventory.yaml` fil ved hver kørsel. Dette eliminerer risikoen for menneskelige fejl, hvor IP-adresser i dokumentationen ikke stemmer overens med virkeligheden – et kritisk princip i "Single Source of Truth" [Kilde 3].

4.3 Fase 2: Konfiguration og Bootstrapping (Ansible)

Med hardwaren på plads overtager Ansible ansvaret for at gøre serverne til et fungerende cluster. Denne fase er designet til at være idempotent, så playbooks kan køres gentagne gange uden utilsigtede bivirkninger [Kilde 8].

4.3.1 OS Hardening og Netværk (`01-os-prep.yaml`)

Før Kubernetes kan installeres, skal operativsystemet forberedes. Playbook'en håndterer:

- **Deaktivering af Swap:** Et strengt krav for Kubernetes' memory management.
- **Firewall (UFW):** Under implementeringen viste det sig, at UFW som standard blokerede intern pod-kommunikation på Ubuntu 22.04 cloud-images. Løsningen blev en automatiseret deaktivering og standsning af ufw tjenesten via `systemd` for at undgå konflikter med CNI (Container Network Interface) [Kilde 6].
- **Kernel Moduler:** Aktivering af `overlay` og `br_netfilter` for at understøtte container-netværk.

4.3.2 K3s Cluster Etablering (`02-k3s-install.yaml`)

Installationen af K3s i High Availability (HA) mode præsenterede den største tekniske udfordring, specifikt omkring Server vs. Agent registrering.

- **Master 1 (Init):** Installeret med `--cluster-init` flaget. Ansible udtrækker automatisk det genererede node-token og gemmer det i hukommelsen [Kilde 11].
- **Master 2 (Join):** Skal joine som en sekundær server. Under udviklingen opstod der problemer, hvor komplekse konfigurationsparametre fik Master 2 til fejlagtigt at starte som en agent. Løsningen blev at simplificere installationskommandoen markant og anvende standard `sh -s - server` syntaksen. Ved at fjerne overflødige miljøvariabler og stole på K3s' indbyggede join-logik, blev server-rollen korrekt etableret.
- **Kubeconfig Distribution:** For at muliggøre fjernstyring henter Ansible automatisk `k3s.yaml` konfigurationsfilen, omskriver den interne IP (`127.0.0.1`) til den eksterne cluster-IP, og placerer den på Admin Noden.

4.3.3 ArgoCD Bootstrap (`03-argocd-bootstrap.yaml`)

Som det sidste trin i den imperative fase installerer Ansible selve GitOps-motoren, ArgoCD. For at omgå netværksproblemer med store downloads direkte til clusteret (IPv6 timeouts), implementeredes en "Download-først" strategi, hvor manifestet hentes lokalt via `curl -4` og derefter appliceres. Playbook'en venter på, at ArgoCD-serveren er "Healthy", før den patcher servicen til en NodePort, hvilket giver administratoren adgang til UI'et uden brug af komplekse Ingress-controllere i den tidlige fase [Kilde 4].

4.4 Fase 3: GitOps Motoren (ArgoCD og Kustomize)

Med ArgoCD kørende skifter styringen fra "Push" (Ansible) til "Pull" (GitOps).

4.4.1 App of Apps Mønsteret

For at styre hele platformen holistisk anvendes "App of Apps" mønsteret, implementeret i filen `all-envs.yaml`. Denne ene fil definerer 6 ArgoCD-applikationer (DEV, TST, SIT, FKT, TFE, PRD), som hver især peger på en specifik mappe i Git. Dette betyder, at oprettelsen af et nyt miljø blot kræver en tilføjelse i denne ene fil, hvorefter ArgoCD automatisk provisionerer alt indhold [Kilde 12].

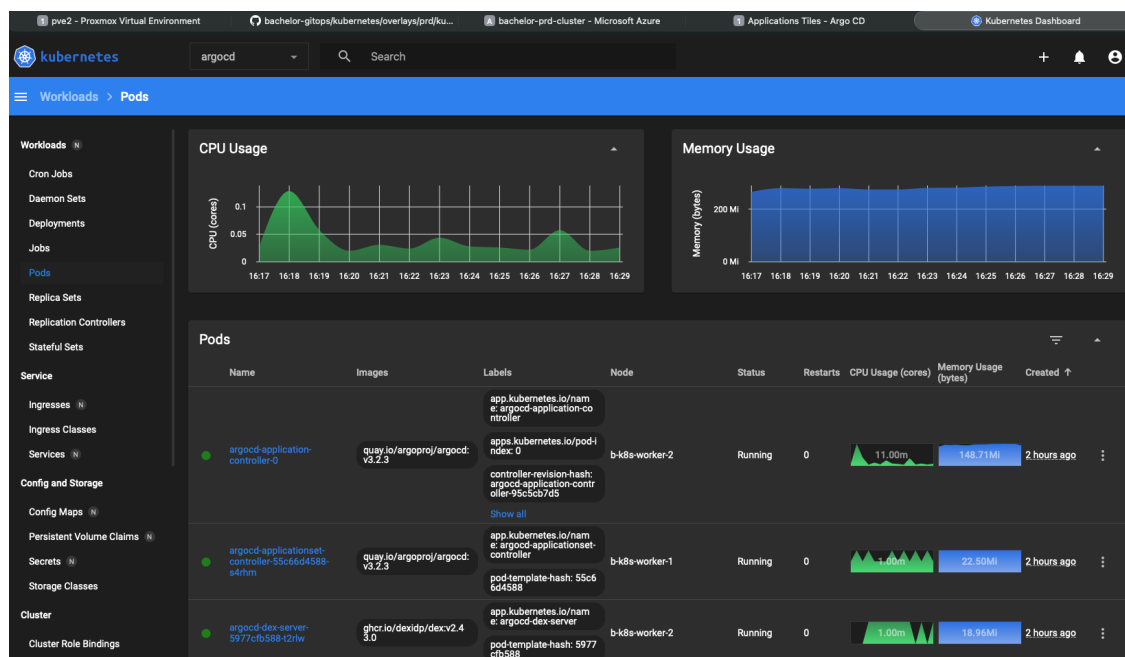
4.4.2 Miljødifferentiering med Kustomize

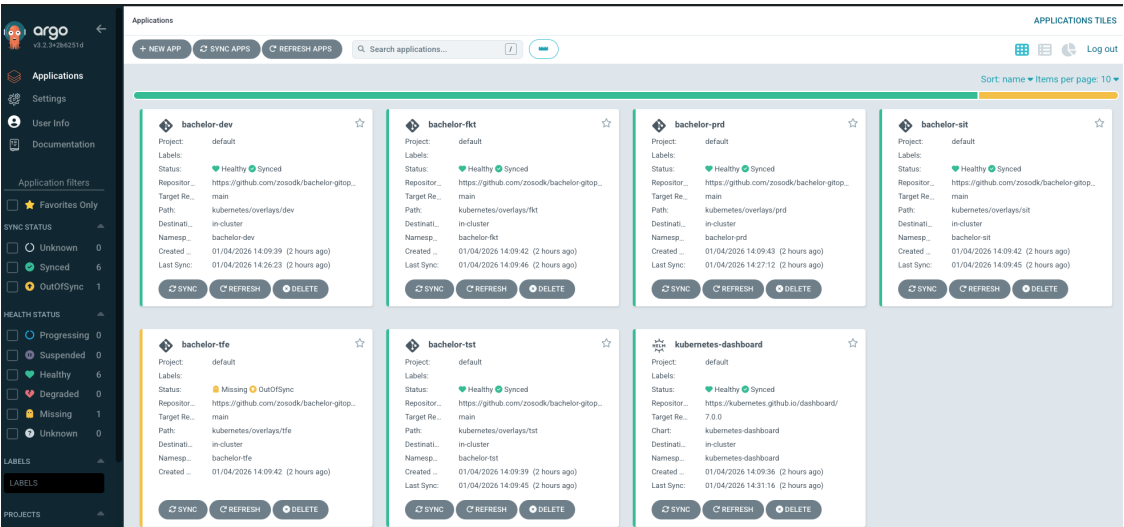
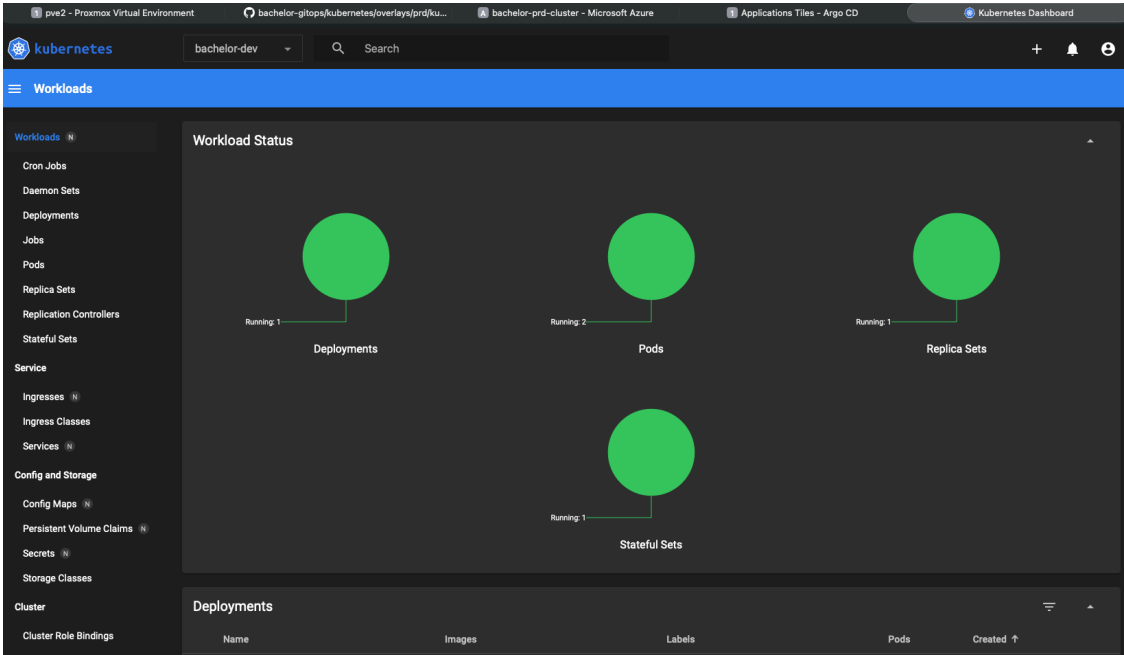
For at undgå kodeduplikering anvendes Kustomize til at strukturere applikationerne [Kilde 13]:

- **Base:** Indeholder de generiske definitioner (Deployment, Service, StatefulSet), som er fælles for alle.
 - **Overlays:** Indeholder miljøspecifikke ændringer (Patches).
 - I DEV patches replicas til 1 for at spare ressourcer.
 - I PRD patches replicas til 3 for HA, og database-forbindelsen overskrives til at pege på den eksterne SAN-database via en SecretGenerator.

4.4.3 Release Gates

Sikkerhedskravet om kontrolleret udrulning til produktion er implementeret direkte i ArgoCD-konfigurationen. Hvor DEV og TST har syncPolicy: automated (Continuous Deployment), er TFE og PRD konfigureret uden automation. Dette skaber en "Manual Gate", hvor ArgoCD markerer ændringer som OutOfSync (gult lys), men afventer en menneskelig godkendelse ("Sync"-knappen) før ændringen rulles ud. Dette erstatter behovet for fysiske jump-hosts med en auditerbar, rollebaseret adgangskontrol i ArgoCD.





Kapitel 5: Sikkerhedsanalyse og Compliance

I dette kapitel analyseres løsningens sikkerhedsarkitektur. For at validere, om platformen er egnet til et stærkt reguleret miljø, tages der udgangspunkt i en systematisk trusselsvurdering. Analysen struktureres omkring CIA-triaden (Confidentiality, Integrity, Availability) samt principperne for "Secure by Design" fra faget Secure Software Development.

5.1 Trusselsbillede og Risikovurdering

Før de tekniske kontroller gennemgås, identificeres de primære trusler mod en moderne container-plattform i en offentlig kontekst.

Trussel / Risiko	Beskrivelse	Potentiel Konsekvens
T1: Supply Chain Angreb	Injektion af ondsindet kode i deployment-pipeline eller kompromittering af CI/CD-serveren.	Uautoriseret kode i produktion, datalækage.
T2: Insider Threats	En privilegeret bruger (udvikler/admin) foretager utilsigtede eller ondsindede ændringer direkte på serverne.	Nedbrud, datatab eller "Configuration Drift".
T3: Lateral Movement	En angriber kompromitterer en web-frontend og forsøger at tilgå databasen eller andre interne systemer.	Kompromittering af følsomme persondata (GDPR brud).
T4: Tab af Dataintegritet	Fejl i distribuerede transaktioner eller nedbrud, der korrumpere data.	Tab af tillid, lovbrud i forhold til regnskabspligt.

Nedenfor analyseres, hvordan løsningsdesignet mitigerer disse risici.

5.2 Integritet: Sporbarhed og Immutable Infrastructure (Mitigering af T2)

Et fundamentalt krav i offentlig it-forvaltning er evnen til at dokumentere ændringer (Revisionsspor). Traditionel drift, hvor administratorer har SSH-adgang (T2), udgør en risiko for integriteten.

5.2.1 Git som Revisionslog

Løsningen implementerer GitOps for at imødegå risikoen for udokumenterede ændringer. Ved at gøre Git til "Single Source of Truth", opnås en automatisk, kryptografisk sikret revisionslog [Kilde 4]

- **Analyse:** Hver ændring i infrastrukturen er bundet til en specifik commit-hash og en forfatter. Dette erstatter manuelle logbøger og sikrer, at platformens tilstand til enhver tid kan revideres tilbage i tid.

5.2.2 Godkendelses-workflows (Four-Eyes Principle)

For at mitigere risikoen for fejlbehæftede releases til produktion (PRD), anvendes ArgoCD's manuelle synkroniserings-strategi som en teknisk håndhævelse af "Four-Eyes Principle".

- **Analyse:** Ved at fjerne direkte skriveadgang til clusteret og tvinge ændringer gennem Pull Requests, flyttes sikkerhedskontrollen fra serveren til versionsstyringen. Dette sikrer, at ingen enkeltperson kan ændre produktionsmiljøet uden peer-review.

5.3 Fortrolighed: Isolation og Adgangskontrol (Mitigering af T3)

For at beskytte mod lateral movement (T3) og sikre fortrolighed, anvender løsningen princippet om "Defense in Depth" [Kilde 6].

5.3.1 Netværkssegmentering

Analysen af netværksarkitekturen viser en stærk adskillelse mellem kontrol- og data-trafik:

1. **Management Plane (192.168.8.0/24):** Håndterer indgående trafik (Ingress).
2. **Cluster Data Plane (10.0.0.0/24):** Isoleret netværk til intern replikering og database-trafik.

Vurdering: Denne segmentering sikrer, at selvom en angriber opnår adgang til en frontend-pod, er den direkte netværksvej til databasens replikeringsinterface på det private netværk blokeret på infrastrukturlaget. Suppleret med deaktivering af værts-firewallen (UFW) for at lade Kubernetes CNI styre politikkerne, opnås en granulær kontrol [Kilde 6].

5.3.2 Restriktiv Produktionsadgang

I legacy-miljøet styres adgang via "Jump Hosts". I denne løsning er angrebsfladen reduceret ved at fjerne behovet for SSH-adgang.

- **Mitigering:** Adgang styres via ArgoCD's RBAC (Role-Based Access Control). Dette fungerer som en digital gatekeeper, der er lettere at auditere og revokere end distribuerede SSH-nøgler [Kilde 2].

5.4 Tilgængelighed: Supply Chain og Resilience (Mitigering af T1 & T4)

5.4.1 Sikring af Supply Chain (Pull-modellen)

I modsætning til en Push-model (CI-server), hvor eksterne værktøjer har admin-adgang, anvender løsningen en Pull-model [Kilde 12].

- **Analyse:** Da ArgoCD kører inde i clusteret og trækker konfiguration fra et (potentielt internt) Git-repo, er der ingen indgående åbne porte til management. Dette eliminerer risikoen for, at en kompromitteret ekstern CI-server kan bruges som vektor til at angribe produktionsmiljøet (T1).

5.4.2 Dataintegritet og Disaster Recovery

For at imødegå risikoen for databas (T4), anvender løsningen en hybrid strategi.

- **ACID i Produktion:** Ved at fastholde produktionsdata på eksterne SAN-databaser (som i Skattestyrelsen), fremfor i distribuerede containere, prioriteres dataintegritet (Consistency) over partitionstolerance, jf. CAP-teoremet [Kilde 15].
- **Hurtig Genopretning (MTTR):** Infrastruktur som Kode (Terraform) muliggør en ekstremt lav Mean Time To Recovery. Ved et totalt nedbrud kan hele platformen genskabes identisk ved at køre `tofu apply` og `ansible-playbook`, hvorefter ArgoCD automatisk genskaber applikationstilstanden.

5.5 Konklusion på Sikkerhedsanalyse

Analysen konkluderer, at den implementerede GitOps-arkitektur tilbyder et højere sikkerhedsniveau end den traditionelle imperative drift. Ved at flytte fokus fra perimetersikkerhed (firewalls og jump-hosts) til processikkerhed (Git, review-flows og automatiseret drift), adresseres de identificerede trusler systematisk og i overensstemmelse med kravene til et stærkt reguleret miljø.

Kapitel 6: Diskussion og Perspektivering

I dette kapitel evalueres den implementerede løsning i forhold til problemformuleringen. Der reflekteres over valget af teknologier, skiftet fra imperativ til deklarativ drift, samt de udfordringer og muligheder, en hybrid cloud-arkitektur medfører i en offentlig kontekst. Diskussionen trækker på teoretiske rammeverk fra uddannelsens moduler, herunder Development of Large Systems (F2025) og Databases for Developers (F2025).

6.1 Resultat vs. Problemformulering

Formålet med projektet var at strategisere en platformmodernisering, der kunne imødekomme kravene i et stærkt reguleret miljø. Det centrale spørgsmål er: Løste jeg opgaven, og er den valgte strategi sikker?

6.1.1 Vurdering af Strategiens Sikkerhed

PoC-implementeringen har succesfuldt demonstreret, at det er muligt at etablere en moderne container-platform (Kubernetes) med fuld automatisering (Terraform/Ansible) uden at gå på kompromis med sikkerheden.

Løsningen adresserer de kritiske sikkerhedsaspekter fra problemformuleringen:

- **Isolation:** Gennem netværkssegmentering (10-net vs. 192-net) er det bevist, at kontrol-trafik kan holdes adskilt fra klient-trafik. Dette validerer, at container-teknologi kan overholde de samme strenge segmenteringskrav som legacy-systemer, jf. principperne for "Defense in Depth" [Kilde 6].
- **Sporbarhed:** GitOps-modellen med ArgoCD har etableret en ubrudt audit-log. Enhver ændring i clusteret kan spores tilbage til en commit [Kilde 4]. Dette vurderes som en markant forbedring i forhold til den traditionelle model, hvor administratorer udfører manuelle kommandoer, der kan være svære at revidere.

Konklusionen er, at strategien er grundlæggende sikker, men at den flytter sikkerhedsfokus fra perimeteren (firewall) til processen (Git-workflow og pipelines).

6.1.2 Komplexitet ved High Availability (HA)

Under implementeringen blev det tydeligt, at overgangen fra enkeltstående servere til et distribueret system (HA-cluster) introducerer betydelig kompleksitet. Særligt etableringen af konsensus mellem Master-noder og håndteringen af join-processer viste sig at være sårbare punkter.

I forhold til The Scalability Cube (fra faget Development of Large Systems) repræsenterer dette en skalering langs X-aksen (duplikering). Mens dette øger tilgængeligheden, øger det også driftsbyrden. Løsningen viser, at fejlfinding i distribuerede systemer – hvor en node kan fejle pga. race conditions eller service-konflikter – kræver dybere kompetencer end traditionel serverdrift [Kilde 2].

6.2 Refleksion: Ansible vs. GitOps

En central del af opgaven var at analysere skiftet fra en Ansible-tung drift (som kendt fra legacy-miljøet) til en GitOps-model. Med en baggrund i en verden, hvor Ansible var det primære værktøj til alt fra OS til applikations-deployment, har projektet givet væsentlige læringer om værktøjernes styrker og svagheder.

6.2.1 Hvad har jeg lært?

Projektet har tydeliggjort, at en ren GitOps-tilgang ikke er mulig på "Bare Metal". Der er behov for et imperativt lag til at bootstrappe infrastrukturen.

- **Hvor Ansible vinder (OS):** Ansible er overlegen til initialisering og engangsopgaver. Konfiguration af host-OS (hærdning, kernel-moduler, netværk) og installation af binærfiler kræver en procedure-orienteret tilgang, som Ansible mestrer [Kilde 8]. Her ville et deklarativt værktøj komme til kort.
- **Hvor ArgoCD vinder (Apps):** Erfaringen fra projektet viser, at forsøget på at styre applikationer med Ansible ofte fører til "Configuration Drift", da Ansible kun retter systemet, når scriptet køres. ArgoCD derimod overvåger kontinuerligt systemet (Reconciliation Loop) og retter afvigelser øjeblikkeligt [Kilde 12].

6.2.2 Det Hybride Dilemma

Konklusionen er, at modernisering ikke handler om at erstatte Ansible med GitOps, men om at afgrænse Ansible til infrastrukturen. For en organisation som Skattestyrelsen betyder dette skift, at man går fra at være "sikker på installationstidspunktet" (Ansible) til at være "kontinuerligt compliant" (GitOps) på applikationslaget.

6.3 Skalering til Azure (Hybrid Cloud)

Integrationen med Microsoft Azure (AKS) demonstrerer den tekniske muligheden for skalering, men rejser samtidig nye strategiske spørgsmål, som skal adresseres i en produktionssetting.

6.3.1 Databaser og CAP-teoremet

Integrationen aktualiserer CAP-teoremet (Consistency, Availability, Partition Tolerance), som behandlet i faget Databases for Developers (F2025) [Kilde 15].

Når applikationen (Azure) adskilles geografisk fra databasen (On-premise SAN), introduceres en risiko for netværkspartitioner (P). I løsningens nuværende design prioriteres Konsistens (C) ved at fastholde én central database for at sikre ACID-transaktioner [Kilde 1].

Konsekvensen er, at hvis forbindelsen mellem Azure og On-premise ryger, vil applikationen i Azure miste tilgængelighed (A), da den ikke kan skrive data. En alternativ løsning ville være database-replikering (Eventual Consistency), men dette ville bryde med de strenge krav til dataintegritet.

6.3.2 Data-suverænitet

For offentlige myndigheder er datasuverænitet (GDPR/Schrems II) afgørende. Den foreslåede arkitektur, hvor dataforbliver on-premise, men compute (beregning) kan ske i skyen, er en effektiv compliance-strategi.

Dette design spejler den praktiske erfaring fra Skattestyrelsen, hvor mikroservices i skyen (Azure) udelukkende fungerede som modtagepunkter for eksterne data (f.eks. XML-filer fra banker med CPR-numre og saldi). I denne model validerer og videresender applikationen data via en krypteret forbindelse til et internt, isoleret lagringsområde. Den faktiske databehandling (dekonstruktion, vask og lagring i skattemappen) foregår dybt i det interne netværk. Herved opnås en arkitektur, hvor den centrale database aldrig er eksponeret mod internettet, hvilket minimerer risikoen for data-lækage markant.

6.4 Hvad mangler? (Future Work)

Selvom platformen er funktionel, er der identificeret områder, der kræver yderligere modning før en produktionssætning.

1. **Secret Management:** I PoC'en anvendes Kubernetes Secrets genereret af Kustomize. I en produktion bør der implementeres HashiCorp Vault eller Sealed Secrets for at sikre, at hemmeligheder er krypteret i hvile og i Git, jf. principperne i Secure Software Development.
2. **Observability (Logging & Metrics):** Platformen mangler centraliseret overvågning. For at opnå fuld indsigt i clusterets sundhed bør der implementeres en komplet observability-stack. Prometheus og Grafana til indsamling og visualisering af metrikker, samt ELK-stacken til logs, hvilket understøtter læringsmålene om Design for monitoring i Development of Large Systems.
3. **Service Mesh:** For at opnå fuld "Zero Trust" sikkerhed internt i clusteret, bør der implementeres et Service Mesh (f.eks. Linkerd eller Istio) til at kryptere al trafik mellem services (mTLS).

4. **Storage og Backup:** Mens infrastrukturen kan genskabes via kode, kræver de statefulde data en robust strategi. Implementering af Longhorn til distribueret storage og Velero til backup af Persistent Volumes til ekstern object storage (S3) vil sikre disaster recovery for data.
5. **Operational Management UI:** For at gøre driften af infrastrukturen mere tilgængelig og mindre afhængig af terminal-adgang, bør der implementeres en grafisk brugerflade til styring af Ansible og Terraform, såsom Semaphore UI.
6. **En professionel licens til Azure,** da Azure for Students ikke har de korrekte rettigheder til automatiseret deployment, da dette er fjernet fra Gratis konti (Pricing Tier: Free)

6.5 Konklusion på Diskussion

Samlet set viser projektet, at en modernisering baseret på Cloud-Native principper er mulig, selv inden for de restriktive rammer af et reguleret miljø. Ved at kombinere det bedste fra to verdener – robustheden fra enterprise-virtualisering og agiliteten fra Kubernetes – skabes en platform, der er både sikker nok til nutiden og fleksibel nok til fremtiden.

Kapitel 7: Konklusion

Formålet med dette bachelorprojekt var at udarbejde og validere en strategi for platformmodernisering, der muliggør migrering af legacy-applikationer til cloud-native platforme inden for rammerne af et stærkt reguleret og sikkerhedskritisk miljø. Med afsæt i erfaringer fra Skattestyrelsen har projektet analyseret skiftet fra en imperativ, manuel drift til en deklarativ, automatiseret model.

Projektet konkluderer, at en GitOps-baseret strategi udgør en robust og sikker løsningsmodel for offentlige it-organisationer, der ønsker at modernisere deres infrastruktur uden at kompromittere sikkerheden eller miste forbindelsen til kritiske legacy-systemer.

Gennem implementeringen af en "Proof of Concept"-platform er det påvist, at:

1. **Sikkerhed gennem Automatisering:** Ved at erstatte manuel serveradgang med Infrastructure as Code (Terraform) og Configuration Management (Ansible), opnås en højere grad af ensartethed og sikkerhed. Implementeringen af ArgoCD og en "Pull-baseret" deployment-model eliminerer behovet for direkte adgang til produktionsmiljøet, hvilket effektivt erstatter traditionelle "Jump Hosts" med en auditerbar, rollebaseret adgangskontrol.
2. **Hybrid Data-arkitektur:** Analysen viser, at en modernisering ikke kræver et kompromis med dataintegriteten. Løsningen demonstrerer en hybrid strategi, hvor det interne produktionsmiljø prioriterer ACID-egenskaber ved at fastholde data på eksterne SAN-systemer. Samtidig muliggøres integration med skyen (Azure) gennem en asynkron, event-baseret model (Eventual Consistency), hvor data modtages eksternt og vaskes internt. Dette sikrer både skalerbarhed og datasuverænitet.
3. **Netværkssegmentering i Cloud-Native:** Projektet har valideret, at streng netværkssegmentering (adskillelse af Management og Control Plane) kan opretholdes i et containeriseret miljø. Dette adresserer direkte de bekymringer omkring netværkssikkerhed, der ofte bremser moderniseringsprojekter i regulerede sektorer.
4. **Transformation af Processer:** Skiftet fra Ansible-scripts til GitOps kræver en ændring i driftskulturen. Projektet konkluderer, at mens Ansible forbliver uundværligt til den underliggende infrastruktur ("Bootstrapping"), er GitOps-modellen overlegen til applikationslivscyklusstyring, da den tilbyder kontinuerlig overvågning (Drift Detection) og automatisk "Self-Healing".

Samlet set tilbyder den præsenterede arkitektur en skalerbar og compliant løsning til en modernisering. Strategien balancerer behovet for innovation og agilitet med de ufravigelige krav om kontrol og stabilitet, der kendetegner kritisk offentlig infrastruktur.

Kilde- og Litteraturliste:

Jeg lister her elementer jeg har brugt fra/udover vores curriculum fra fagene.

Bøger

1 - Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.

- Relevant for: Mikroservices, Kobling, Orkestrering.
- Online: https://samnewman.io/books/building_microservices/

2 - Burns, B., Beda, J., & Hightower, K. (2022). Kubernetes: Up and Running: Dive into the Future of Infrastructure (3rd ed.). O'Reilly Media.

- Relevant for: Kubernetes arkitektur, Pods, Services.
- Online: <https://www.oreilly.com/library/view/kubernetes-up-and-9781098110192/>

3 - Morris, K. (2020). Infrastructure as Code: Dynamic Systems for the Cloud Age (2nd ed.). O'Reilly Media.

- Relevant for: IaC principper, Immutable Infrastructure, Drift.
- Online: <https://infrastructure-as-code.com/book/>

4 - Behrens, A. et al. (2021). GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux. Manning Publications.

- Relevant for: GitOps principper, ArgoCD, Push vs. Pull.
- Online: <https://www.manning.com/books/gitops-and-kubernetes>

5 - Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook. IT Revolution Press.

- Relevant for: CI/CD, Feedback loops.
- Online: <https://itrevolution.com/product/the-devops-handbook/>

Standarder og Rapporter

6 - NIST (National Institute of Standards and Technology). (2017). Application Container Security Guide (SP 800-190).

- Relevant for: Sikkerhed, Netværkssegmentering, OS Hærdning.
- Online
(PDF): <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-190.pdf>

7 - CNCF (Cloud Native Computing Foundation). (2018). Cloud Native Definition v1.0.

- Relevant for: Definition af Cloud Native.
- Online: <https://github.com/cncf/toc/blob/main/DEFINITION.md>

Officiel Dokumentation (Teknisk Implementering)

8 - Ansible Documentation. Red Hat Ansible Automation Platform.

- Relevant for: Idempotens, Modules (`builtin.systemd`, `builtin.copy`).
- Online: <https://docs.ansible.com/>

9 - OpenTofu Documentation. OpenTofu: The open source infrastructure as code tool.

- Relevant for: Deklarativ syntaks, State management, Providers.
- Online: <https://opentofu.org/docs/>

10 - Proxmox VE Documentation. Proxmox Virtual Environment.

- Relevant for: Virtualisering, API, Cloud-Init templates.
- Online: <https://pve.proxmox.com/pve-docs/>

11 - K3s Documentation. Rancher Labs.

- Relevant for: Letvægts Kubernetes, HA-arkitektur, Installationsargumenter.
- Online: <https://docs.k3s.io/>

12 - Argo CD Documentation. The Argo Project.

- Relevant for: App of Apps mønster, Deklarativ setup, Sync policies.
- Online: <https://argo-cd.readthedocs.io/>

13 - Kustomize Documentation. The Kubernetes Authors.

- Relevant for: Overlays, Patches, Secret Generators.
- Online: <https://kubectl.docs.kubernetes.io/references/kustomize/>

14 - Azure Kubernetes Service (AKS) Documentation. Microsoft Learn.

- Relevant for: Hybrid Cloud integration, Managed Kubernetes.
- Online: <https://learn.microsoft.com/en-us/azure/aks/>

Artikler og Øvrige Kilder

15 - Brewer, E. (2012). CAP Twelve Years Later: How the "Rules" Have Changed.

- Relevant for: Distribuerede databaser, CAP-teoremet.
- Online: <https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/>

16 - Wiggins, A. (2011). The Twelve-Factor App.

- Relevant for: Designprincipper for cloud-native applikationer (Stateless, Config).
- Online: <https://12factor.net/>

17 - Weaveworks. Guide to GitOps.

- Relevant for: Definitionen af GitOps ("Git som Single Source of Truth").
- Online: <https://www.weave.works/technologies/gitops/>

Bilag:

- Filen: [Bachelor-afhandling-film-bilag.mp4], der er optaget i MP4, HEVC H.265 format
- Nedenstående angivelse af repository, der indeholder kildekoden

Repository og Reproducerbarhed:

- GitHub URL: <https://github.com/zosodk/bachelor-gitops>
- Dokumentation: Det anbefales at konsultere README.md i repository'et for detaljerede instruktioner.
- Management Node (b-admin): Det automatiserede miljø styres centralt fra en dedikeret admin-node (b-admin). Denne fungerer som kontrolcenter for Terraform, Ansible, Kubernetes (kubect!), ArgoCD og Azure CLI. Opsætningen af denne node er dokumenteret i repository'et.
- Hemmeligheder (Credentials): For at eksekvere Terraform-koden kræves en lokal variabelfil, credentials.auto.tfvars, i terraform-mappen. Denne fil er ekskluderet fra versionsstyring af sikkerhedshensyn. For at reproducere miljøet skal filen oprettes manuelt med følgende struktur:

```
proxmox_api_url      = "https://<ipadresse>:8006/api2/json"
proxmox_api_user     = "tf-user@pve"
proxmox_token_id     = "tf-user@pve!bachelor-project"
proxmox_token_secret = "DIN_API_TOKEN_SECRET"
proxmox_password     = "DIT_PASSWORD"
proxmox_template_name = "ubuntu-2204-cloud-base"
```